AR/VR ASSIGNMENT – 4

PLANETARY SYSTEMS & ORBITAL MECHANICS

OPTION 1:

Static Unity 3D Planetary Simulation

Objective:

The objective of this project was to create a high-fidelity 3D planetary simulation using the Unity Engine. The project demonstrates a simplified solar system environment containing the Sun, Earth, and Moon with realistic textures, lighting, and planetary rotation.

Introduction:

This project was developed using Unity 6 and focuses on creating a realistic planetary visualization. Different celestial bodies were modelled using 3D spheres and enhanced with realistic textures. A space-themed skybox background and dynamic lighting were added to improve the visual quality of the scene.

The project demonstrates basic concepts of planetary systems, scene composition, material mapping, and animation within Unity.

Technologies Used:

- Unity 6
- C#
- Universal Render Pipeline (URP)
- NASA / Planetary Texture Maps

Scene Design:

The planetary system was created using Unity primitive spheres representing the Sun, Earth, and Moon. Each object was scaled and positioned manually inside the 3D environment to simulate a simplified solar system structure.

A space skybox was added as the environment background to create an immersive outer-space appearance.

Texture Mapping:

High-resolution textures were imported and applied to the celestial bodies using Unity materials.

Separate materials were created for:

- Sun
- Earth
- Moon

These textures improved realism by displaying planetary surface details and lighting effects.

Lighting and Environment:

A directional light and point light were used to illuminate the planetary scene. The Sun acts as the primary light source, creating realistic shading and reflections on nearby planets.

The camera was positioned strategically to provide a cinematic view of the planetary setup.

Planet Rotation:

A custom C# script was implemented to simulate continuous planetary rotation.

The script uses Unity's:

- Update() function
- transform.Rotate() method

to animate the planets in real time.

Features Implemented:

- Realistic Earth texture
- Moon texture mapping
- Sun texture and glow
- Space skybox background
- Planet rotation animation
- Cinematic camera positioning
- 3D planetary visualization in Unity

Results:

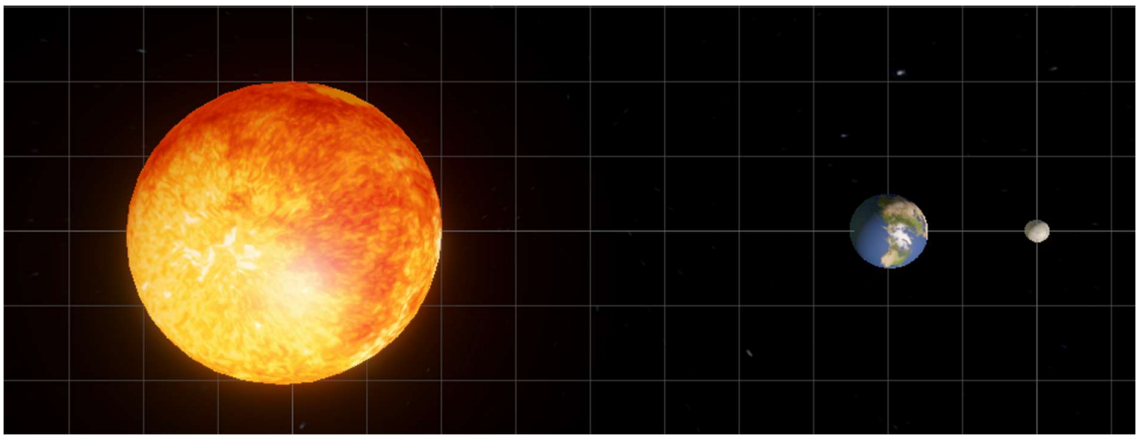
The final output successfully displays a realistic 3D planetary scene with textured celestial bodies and rotational animation. The simulation demonstrates the fundamentals of planetary visualization and scene development in Unity.

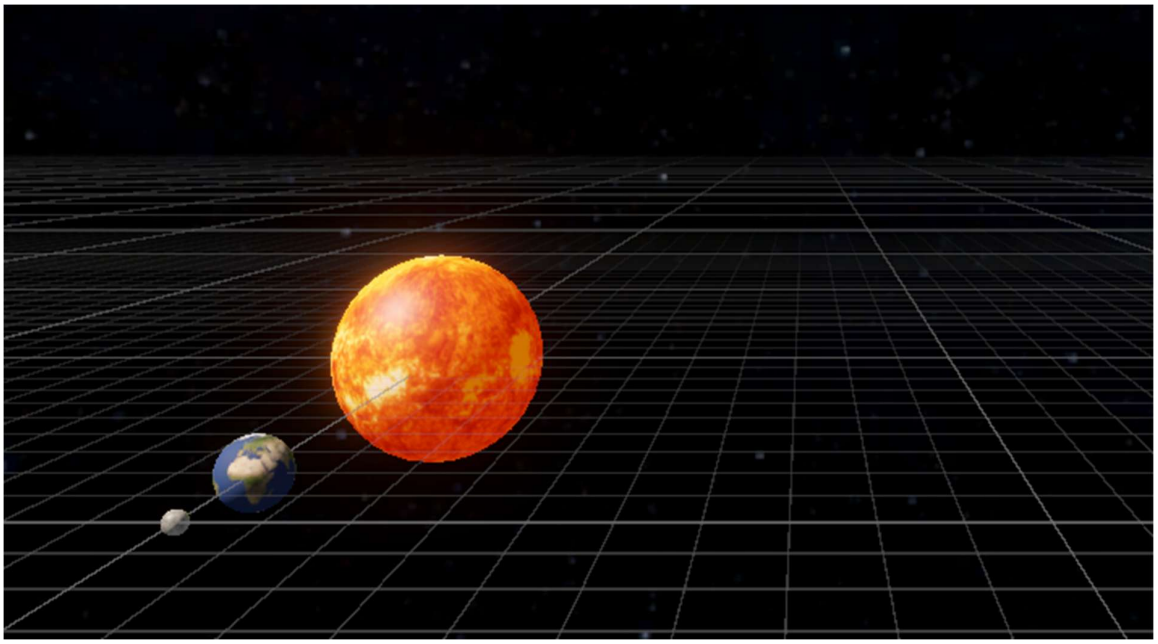
Conclusion:

This project successfully demonstrates a static planetary simulation using Unity Engine. The implementation includes realistic textures, lighting, and planetary rotation to create an immersive space environment.

The assignment helped in understanding Unity scene creation, texture mapping, lighting systems, and basic animation concepts used in planetary simulations.

Screenshots:





OPTION 2:

WebXR Earth-Moon Orbital System

Objective:

The objective of this project was to create a dynamic Earth-Moon orbital simulation using Unity Engine concepts inspired by WebXR and A-Frame orbital mechanics. The project demonstrates realistic planetary motion where the Earth rotates on its own axis while the Moon simultaneously rotates and revolves around the Earth in a circular orbit.

The simulation was designed to visualize orbital mechanics in a 3D space environment with realistic textures, lighting, and smooth animations.

Technical Approach:

A separate Unity scene named **Orbital System** was created for the implementation of the Earth-Moon animation system.

The project includes:

- Earth sphere with realistic Earth texture
- Moon sphere with Moon surface texture
- Space skybox background
- Continuous Earth rotation
- Continuous Moon rotation
- Moon revolution around Earth using parent-child hierarchy
- Directional lighting and space environment setup

The Moon was placed inside an empty GameObject called **MoonOrbit.** By rotating the MoonOrbit object, the Moon revolves around the Earth while still rotating independently on its own axis.

Technologies Used:

- Unity 6
- C#
- Universal Render Pipeline (URP)
- 3D Sphere Objects
- Space Skybox Materials
- Planetary Textures

Scene Setup:

The orbital system scene contains:

- Main Camera
- Directional Light
- Earth Object
- MoonOrbit Empty Object
- Moon Object
- Space Skybox Background

The Earth was positioned at the center of the scene while the Moon was positioned at a fixed distance from the Earth inside the MoonOrbit object.

Rotation and Orbit Animation:

Two different motion systems were implemented:

Earth Rotation:

The Earth rotates continuously around its Y-axis using a custom C# script.

Moon Rotation:

The Moon also rotates continuously around its own axis.

Moon Revolution:

The Moon revolves around the Earth by rotating the parent MoonOrbit object around the Y-axis.

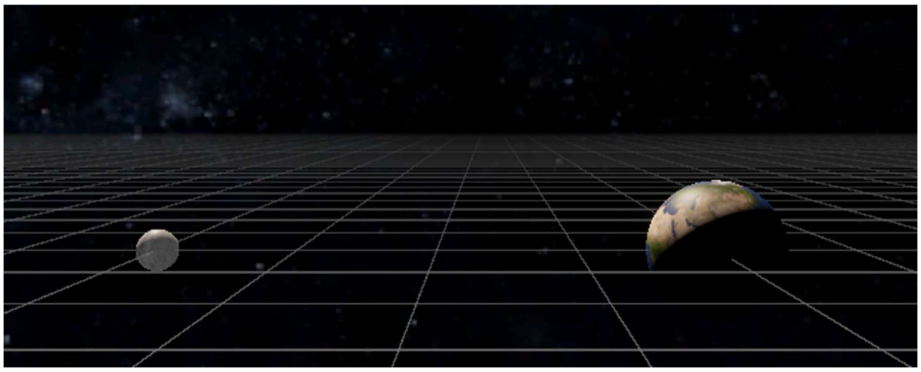
This hierarchy-based animation system creates a realistic orbital effect.

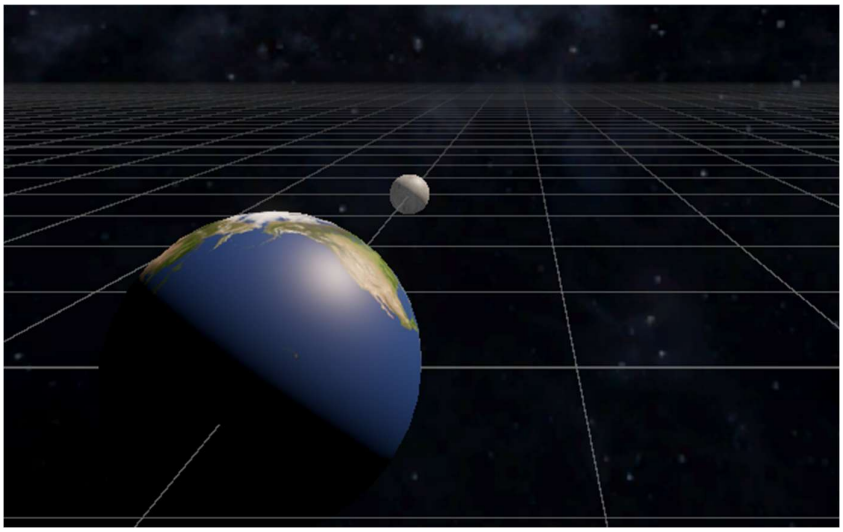
Lighting and Environment:

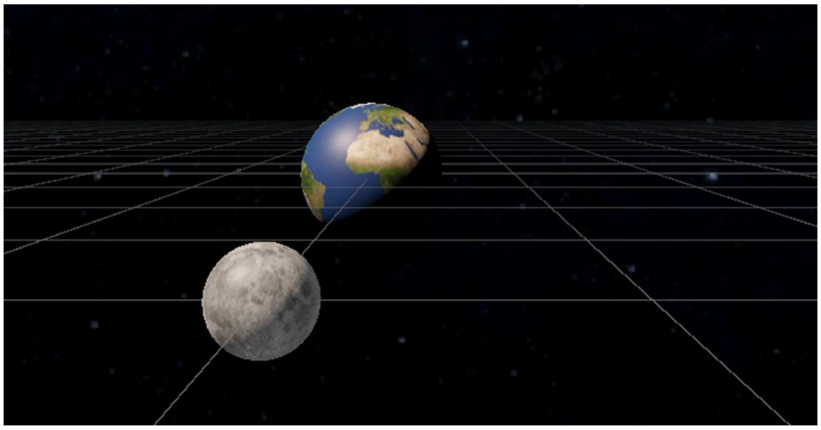
A space-themed skybox was used to simulate the outer-space environment. Directional lighting and point lighting were added to improve visibility and realism of the planetary textures.

The Sun object was configured using emission materials to create a glowing appearance from all directions.

Visual Evidence:







Conclusion:

This project successfully demonstrates orbital mechanics using Unity Engine. The Earth rotates continuously while the Moon rotates and revolves around the Earth simultaneously. The addition of realistic textures, lighting, and space background improved the visual quality of the simulation and provided a better understanding of planetary motion in a 3D environment.